

CmdTrace Multi-Platform Plan

CONTENTS

01 1. 전략 요약

02 2. 프론트엔드 / 백엔드 구분

03 3. 프로젝트 구조

04 4. 구현 계획

05 5. 사용성 비교 항목

06 6. 코드 공유 전략

07 7. 일정 요약

08 8. 리스크 & 대응

09 9. 성공 기준

CmdTrace Multi-Platform Plan

Swift 앱 유지 + Webapp 대시보드 추가 + Tauri 버전 사용성 비교

1. 전략 요약

...

현재 목표

Swift macOS 앱 (v2.4.1) — Swift 앱 + 내장 HTTP 서버 (API 제공)

|

└─ Webapp 대시보드 (브라우저, React)

|

└─ Swift 내장 서버에서 데이터 수신

|

└─ Tauri 앱 (독립 빌드, 사용성 비교용)

|

└─ Rust 백엔드 (파일 파싱)

|

└─ React 프론트 (Webapp과 코드 공유)

...

핵심 결정사항

항목	결정
Webapp 데이터 접근	Swift 앱 내장 로컬 HTTP 서버
UI 프레임워크	React + Tailwind CSS
Webapp ↔ Tauri 코드 공유	공통 React 컴포넌트 패키지
Tauri 백엔드	Rust (파일 파싱, 검색)

2. 프론트엔드 / 백엔드 구분

프론트엔드 (사용자가 보는 화면)

...

공유 React 컴포넌트 (@cmdtrace/ui)

└─ SessionList 세션 목록

- └─ ConversationView 대화 내용 표시
- └─ SearchBar 검색 (operators 지원)
- └─ TagManager 태그 관리
- └─ UsageDashboard 사용량 차트
- └─ BurnRateChart 소진율 그래프
- └─ SessionDiff 세션 비교
- └─ SettingsPanel 설정
- ...

백엔드 (데이터 처리, 눈에 안 보이는 엔진)

기능	SWIFT 앱	TAURI (RUST)
JSONL 파싱	Swift + 내장 서버 API	Rust <code>serde_json</code>
파일 감시	FSEvents → WebSocket 푸시	<code>notify</code> crate
검색	메모리 내 필터 → REST API	Rust 메모리 필터 → IPC
ccusage 실행	<code>Process()</code> → API 응답	<code>Command::new()</code> → IPC
터미널 Resume	AppleScript	<code>shell_execute</code>
AI API 호출	URLSession → 프록시 API	<code>reqwest</code> crate
메타데이터 저장	JSON 파일	JSON 파일 (Rust fs)

3. 프로젝트 구조

...

~/DEV/

- └─ CmdTrace/ ← 현재 Swift 앱 (유지)
 - └─ Sources/
 - └─ Services/
 - └─ LocalServer.swift ← 새로 추가: 내장 HTTP 서버
 - └─ ...
- └─ cmdtrace-web/ ← 새 프로젝트: 공유 프론트엔드 + Webapp
 - └─ packages/
 - └─ ui/ ← 공유 React 컴포넌트
 - └─ src/
 - └─ components/

```

| | | |—— hooks/
| | | |—— types/
| | |—— package.json
| |—— apps/
| |—— dashboard/ ← Webapp 대시보드
| |—— src/
| |—— index.html
| |—— package.json
|—— package.json ← monorepo root
|—— turbo.json
|
|—— cmdtrace-tauri/ ← 새 프로젝트: Tauri 앱
|—— src/ ← React 프론트 (ui 패키지 사용)
|—— src-tauri/ ← Rust 백엔드
|—— src/
| |—— main.rs
| |—— parser.rs ← JSONL 파싱
| |—— search.rs ← 검색 엔진
| |—— watcher.rs ← 파일 감시
| |—— commands.rs ← IPC 핸들러
|—— Cargo.toml
|—— package.json
...

```

4. 구현 계획

Phase A: Swift 내장 서버 + API (Week 1)

Swift 앱에 로컬 HTTP 서버를 추가하여 세션 데이터를 API로 노출.

A-1. 내장 HTTP 서버

항목	내용
라이브러리	<code>NIOCore</code> + <code>NIOHTTP1</code> (SwiftNIO) 또는 경량 <code>Swifter</code>
포트	<code>localhost:19840</code> (고정, 설정 가능)
시작/종지	앱 설정에서 토글
CORS	<code>localhost</code> origin 허용

A-2. REST API 설계

...

GET /api/sessions 세션 목록 (필터, 정렬, 페이지네이션)

GET /api/sessions/:id 세션 상세 (메시지 포함)

GET /api/sessions/:id/messages 메시지 목록 (페이지네이션)

GET /api/search?q=... 검색

GET /api/tags 태그 목록

POST /api/tags 태그 생성

PUT /api/tags/:id 태그 수정

GET /api/metadata 앱 메타데이터 (설정, 통계)

GET /api/usage 사용량 데이터 (ccusage)

WS /ws/events 실시간 이벤트 (새 세션, 파일 변경)

...

A-3. 작업 목록

- ☐ Swift 내장 HTTP 서버 구현 (LocalServer.swift)
- ☐ REST API 엔드포인트 구현
- ☐ WebSocket 이벤트 스트림 구현
- ☐ 앱 설정에 서버 토글 UI 추가
- ☐ CORS 설정

Phase B: Webapp 대시보드 (Week 1-2)

React + Tailwind로 브라우저 대시보드 구현. Swift API에서 데이터 수신.

B-1. 기술 스택

항목	선택
프레임워크	React 19 + TypeScript
스타일	Tailwind CSS v4
빌드	Vite
차트	Recharts 또는 Chart.js
마크다운	react-markdown + rehype
상태 관리	Zustand
HTTP	fetch (내장)
실시간	WebSocket (내장)
모노레포	Turborepo

B-2. 페이지 구성

페이지	기능	API
/	대시보드 (통계 요약, 최근 세션)	GET /api/metadata, /api/sessions
/sessions	세션 목록 + 검색	GET /api/sessions, /api/search
/sessions/:id	세션 상세 (대화 뷰)	GET /api/sessions/:id
/tags	태그 브라우저	GET /api/tags
/usage	사용량 모니터링	GET /api/usage
/settings	연결 설정	-

B-3. 작업 목록

- ☐ monorepo 셋업 (Turborepo + pnpm)
- ☐ 공유 UI 패키지 (@cmdtrace/ui) 초기화
- ☐ API 클라이언트 훅 구현 (useSession , useSearch 등)
- ☐ 타입 정의 (Session, Message, Tag 등)
- ☐ 대시보드 페이지
- ☐ 세션 목록 + 검색 페이지
- ☐ 세션 상세 (대화 뷰) 페이지
- ☐ 사용량 모니터링 페이지
- ☐ WebSocket 실시간 업데이트
- ☐ 다크모드 지원

Phase C: Tauri 앱 (Week 2-3)

공유 React 프론트엔드 + Rust 백엔드로 독립 데스크톱 앱 구현.

C-1. 기술 스택

항목	선택
Tauri	v2
Rust 백엔드	serde_json, notify, tokio, reqwest
프론트엔드	@cmdtrace/ui 공유 패키지 재사용
IPC	Tauri Commands (invoke)

C-2. RUST 백엔드 모듈

```
```rust
// parser.rs - JSONL 파일 파싱
pub fn parse_sessions(path: &Path) -> Vec<Session>
pub fn parse_messages(path: &Path) -> Vec<Message>

// search.rs - 검색 엔진
pub fn search(sessions: &[Session], query: &str) -> Vec<Session>
pub fn parse_operators(query: &str) -> SearchQuery

// watcher.rs - 파일 감시
pub fn watch_sessions(paths: Vec<PathBuf>, tx: Sender<Event>)

// commands.rs - Tauri IPC 핸들러
#[tauri::command]
fn get_sessions() -> Vec<Session>
#[tauri::command]
fn get_session(id: String) -> Session
#[tauri::command]
fn search_sessions(query: String) -> Vec<Session>
...
```
```

C-3. 프론트엔드 어댑터 패턴

Webapp과 Tauri에서 같은 React 컴포넌트를 쓰되, 데이터 소스만 다르게:

```
```typescript
// 공유 인터페이스
interface SessionAPI {
 getSessions(): Promise<Session[]>
 getSession(id: string): Promise<Session>
 search(query: string): Promise<Session[]>
}
```

```
// Webapp 구현: HTTP fetch

class WebAPI implements SessionAPI {

 async getSessions() {

 return fetch('http://localhost:19840/api/sessions').then(r => r.json())

 }

}

// Tauri 구현: IPC invoke

class TauriAPI implements SessionAPI {

 async getSessions() {

 return invoke('get_sessions')

 }

}

...

```

#### C-4. 작업 목록

- ☐ Tauri v2 프로젝트 초기화
- ☐ Rust: JSONL 파서 구현
- ☐ Rust: 세션 검색 엔진
- ☐ Rust: 파일 감시 (notify)
- ☐ Rust: Tauri IPC 커맨드
- ☐ 프론트: @cmdtrace/ui 통합
- ☐ 프론트: Tauri API 어댑터
- ☐ 메타데이터 저장 (tags, favorites)
- ☐ 다크모드 + 시스템 연동
- ☐ 빌드 & 테스트

---

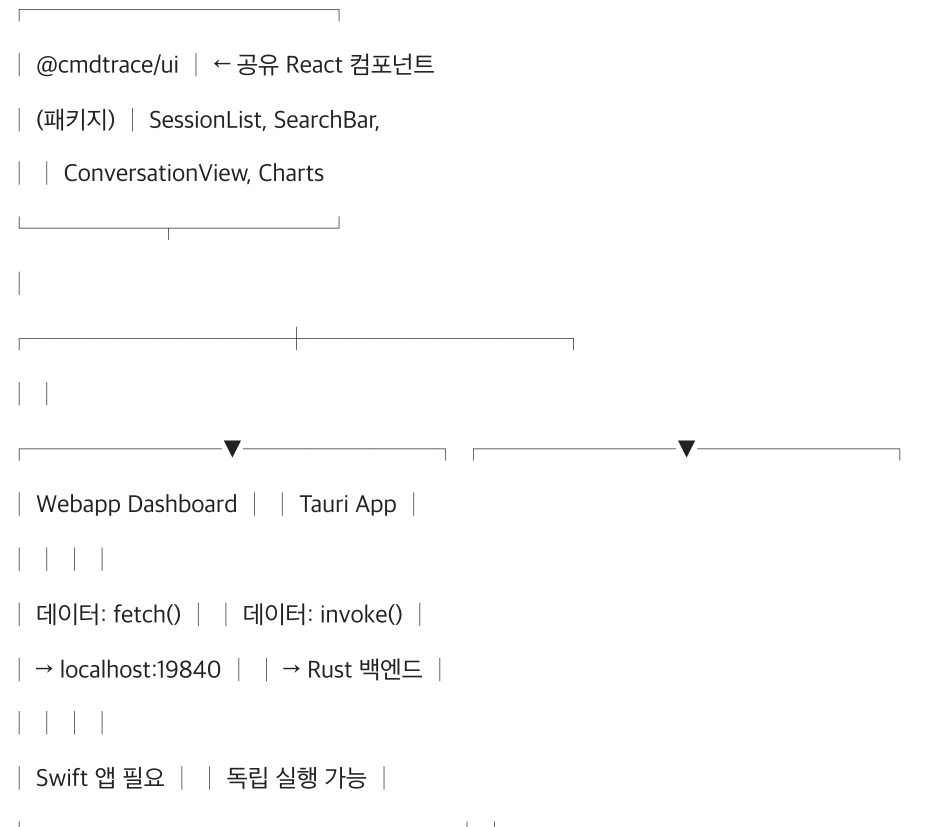
## 5. 사용성 비교 항목

세 버전을 나란히 비교할 체크리스트:

비교 항목	SWIFT	WEBAPP	TAURI
앱 시작 시간	측정	측정	측정
세션 로딩 시간 (100개)			
세션 로딩 시간 (1000개)			
검색 응답 시간			
메모리 사용량 (유틸)			
메모리 사용량 (활성)			
앱 번들 크기			
UI 반응성 (체감)			
네이티브 느낌 (1-5)			
마크다운 렌더링 품질			
검색 연산자 지원			
다크모드 전환			
실시간 업데이트			

## 6. 코드 공유 전략

...



...

공유 범위:

- ☐ ☒ React 컴포넌트 (UI)
- ☐ ☒ TypeScript 타입 정의
- ☐ ☒ 유틸리티 함수 (날짜 포맷, 검색 파서)
- ☐ ☒ 데이터 페칭 로직 (플랫폼별 어댑터)
- ☐ ☒ 백엔드 로직 (Swift vs Rust)

7. 일정 요약

...

Week 1 Week 2 Week 3 Week 4

[Phase A: Swift 내장 서버 ———]

[Phase B: Webapp 대시보드 —————]

[Phase C: Tauri 앱 —————]

[비교 테스트]

...

PHASE	기간	산출물
A	3-4일	Swift 앱 + REST API + WebSocket
B	5-7일	Webapp 대시보드 (브라우저)
C	7-10일	Tauri 데스크톱 앱
비교	2-3일	사용성 비교 리포트

8. 리스크 & 대응

리스크	영향	대응
SwiftNIO 복잡도	서버 구현 지연	경량 대안 <b>Swifter</b> 사용
Rust 학습 곡선	Tauri 구현 지연	AI 코딩 보조 적극 활용
컴포넌트 공유 호환성	코드 중복 발생	어댑터 패턴 철저히 적용
포트 충돌	서버 시작 실패	자동 포트 탐색 fallback

## 9. 성공 기준

- ☐ Swift 앱에서 서버 토글 시 Webapp 대시보드 즉시 접속 가능
- ☐ Webapp에서 세션 검색, 대화 보기, 사용량 확인 가능
- ☐ Tauri 앱이 독립적으로 세션 파싱 및 표시 가능
- ☐ 세 버전의 사용성 비교 데이터 수집 완료
- ☐ React 컴포넌트 60% 이상 Webapp ↔ Tauri 공유